

① Dependencies

- o `sqlite` - runs `sqlite` engine on device, enables local storage and `sql` queries offline.
- o `path` - finds the correct folder to save the `.db` file on each platform

② Database Exception Handling

- Not initialized: use a single getter that checks if the database is open before every query. if the file missing, `sqlite` recreates it automatically
- Insert / update / delete errors: wrap each in `try-catch`. validation errors (empty title, null field) are shown to the user. system errors (storage full) show a friendly message instead of a raw crash.
- Corrupted data: parse each row inside `try-catch` and return null if it fails. filter out nulls before displaying the list so one bad row doesn't crash the whole screen.

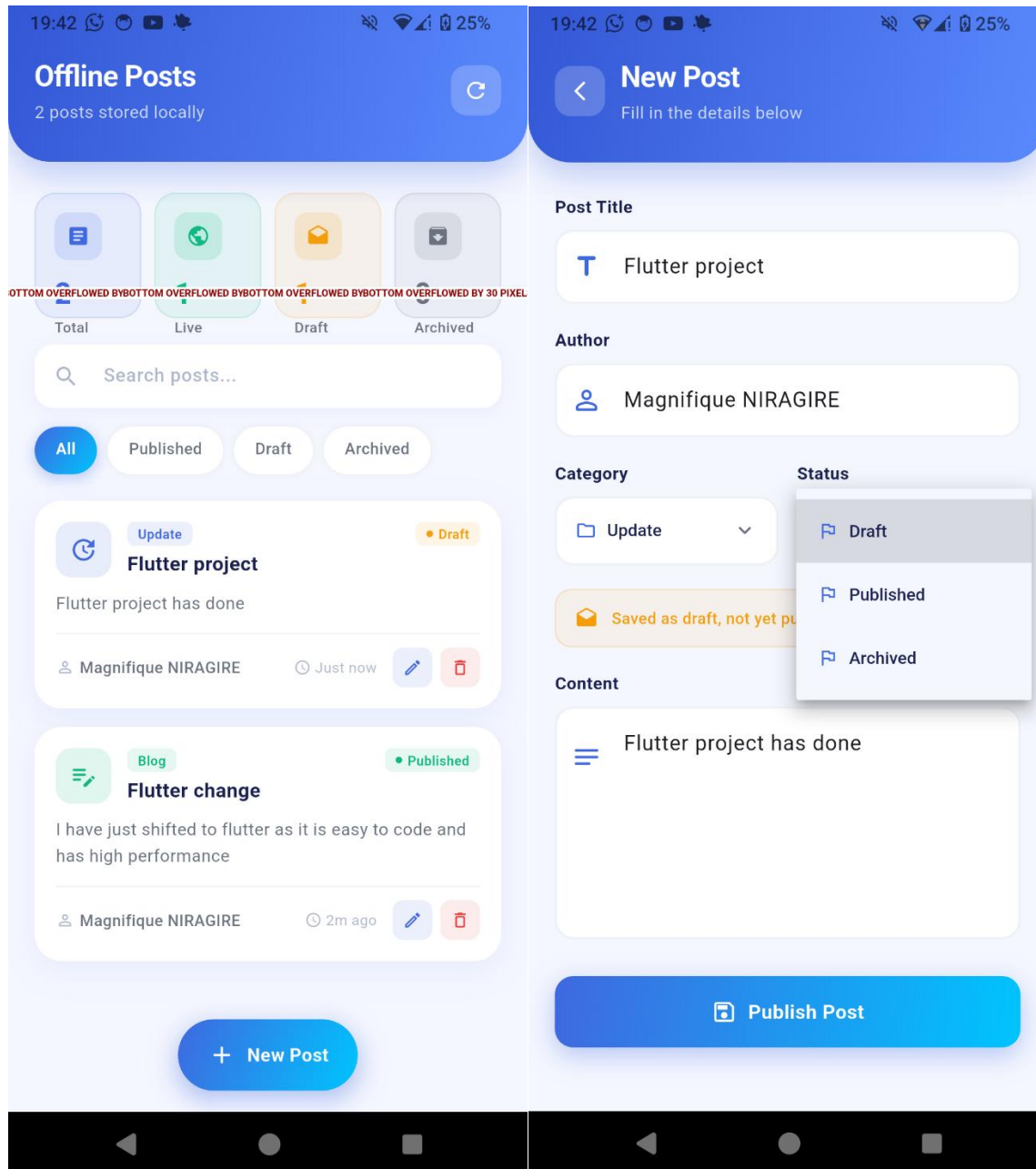
2) `SQLite` in flutter

Database vs Table - the database is one `.db` file on disk (the whole workbook). A table is a structured sheet inside it with typed columns. This app has one database, one table called `posts`

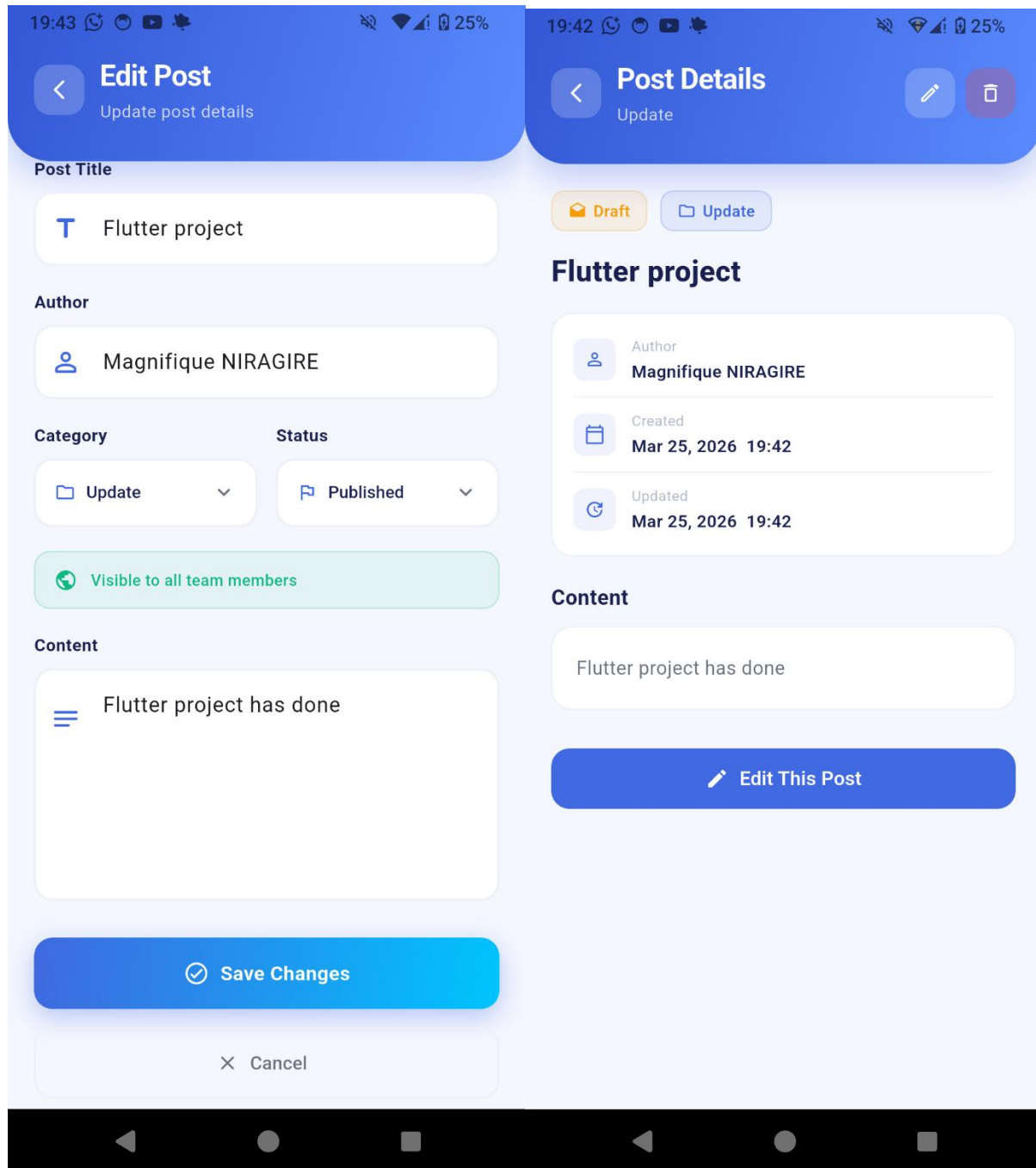
CRUD

- o create - `db.insert()` adds a new row, returns the auto-generated ID
- o Read - `db.query()` with `where` and `orderBy` returns a list of maps converted to post objects
- o update - `db.update()` with a `where` clause targeting the row ID
- o Delete - `db.delete()` with a `where` clause targeting the row ID

Screenshots: Landing page and create a new post



Edit Post and View Post



Delete Post

